



Sistemas Embebidos I

Otoño 2026

Práctica 2: Bitwise & GPIO

Camila Rodríguez Rosas

Ana Karen Rojas Ledesma

Jonathan Hernández Lazcano

Viernes 28 de agosto de 2026

Índice

Práctica 2: Bitwise & GPIO	2
Introducción	2
Descripción de la práctica.....	2
Objetivo.....	2
Marco teórico.....	2
Desarrollo	3
Materiales	3
Esquemático	4
Desarrollo	5
Resultados	7
Evidencias.....	7
Conclusiones	8
Declaración de IA.....	9
Referencias	9

Práctica 2: Bitwise & GPIO

Introducción

Descripción de la práctica

En esta práctica se implementa un sistema de antirrebote (debouncing) por software para un botón mecánico conectado a un microcontrolador Raspberry Pi Pico, utilizando una Máquina de Estados Finitos (FSM, por sus siglas en inglés) como mecanismo de control. El sistema consta de cuatro estados (ST_RELEASED, ST_PRESS_DEBOUNCE, ST_PRESSED, ST_RELEASE_DEBOUNCE) que permiten distinguir una pulsación real del botón de las conmutaciones espurias generadas por el rebote mecánico de sus contactos. La práctica se desarrolla en dos variantes de programación: una utilizando las funciones de alto nivel del SDK de C/C++ para la Raspberry Pi Pico, y otra accediendo directamente a los registros de hardware (SIO, IO_BANK0 y PADS_BANK0), con el fin de comparar ambos enfoques de desarrollo embebido.

Objetivo

Diseñar en SDK y utilizando acceso directo a registros (SIO), una máquina de estados finitos capaz de filtrar el rebote mecánico de un botón conectado a un GPIO de la Raspberry Pi Pico, de modo que cada pulsación física del botón produzca exactamente un cambio de estado en un LED indicador, validando su correcto funcionamiento mediante pruebas de pulsación simple, sostenida y múltiple

Marco teórico

Entradas digitales. Un GPIO configurado como entrada digital permite al microcontrolador leer un nivel lógico —alto (1) o bajo (0)— proveniente de una señal externa, como un botón o sensor, sin imponer voltaje sobre la línea. Cuando un pin de entrada no está conectado a nada, su nivel lógico no está definido: se dice que "flota", pudiendo leer valores aleatorios por efecto del ruido ambiental.

Resistencias de pull-up y pull-down. Para fijar un nivel lógico por defecto cuando el botón no está presionado, se emplean resistencias hacia VDD (*pull-up*) o hacia GND

(*pull-down*). La Raspberry Pi Pico cuenta con resistencias internas activables por software mediante las funciones `gpio_pull_up()` y `gpio_pull_down()`. En una configuración con pull-up, el pin permanece en estado alto en reposo y cambia a bajo únicamente cuando el botón conecta la línea a tierra al presionarse.

Rebote (bounce). Al presionar o soltar un botón mecánico, sus contactos metálicos no se tocan ni se separan de forma limpia e instantánea, sino que generan múltiples conmutaciones rápidas en una ventana de aproximadamente 1 a 20 ms. Dado que el microcontrolador puede muestrear la entrada mucho más rápido de lo que tarda el contacto en estabilizarse, este fenómeno provoca lecturas erróneas, como registrar varios cambios de estado a partir de una sola pulsación física.

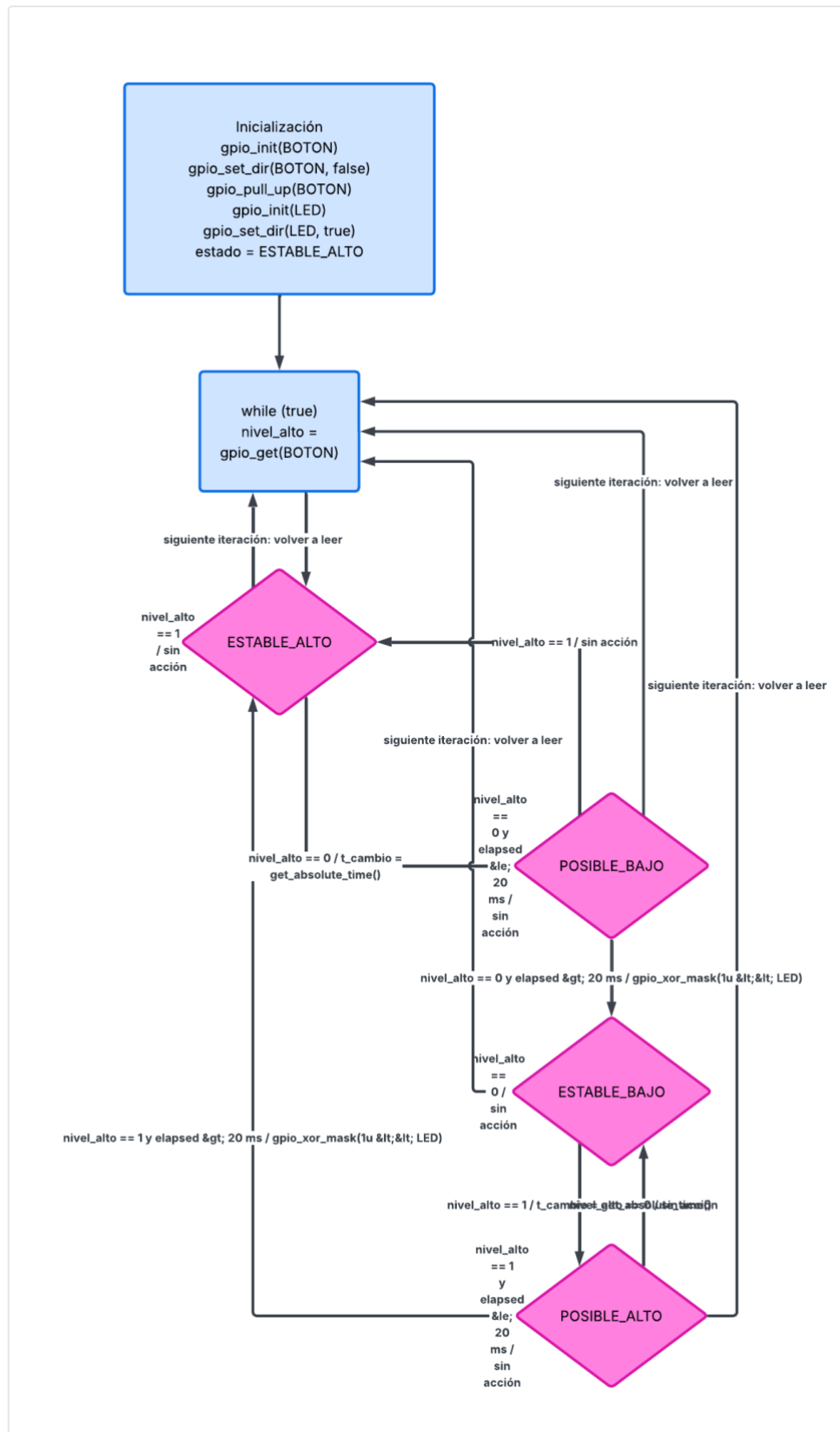
Desarrollo

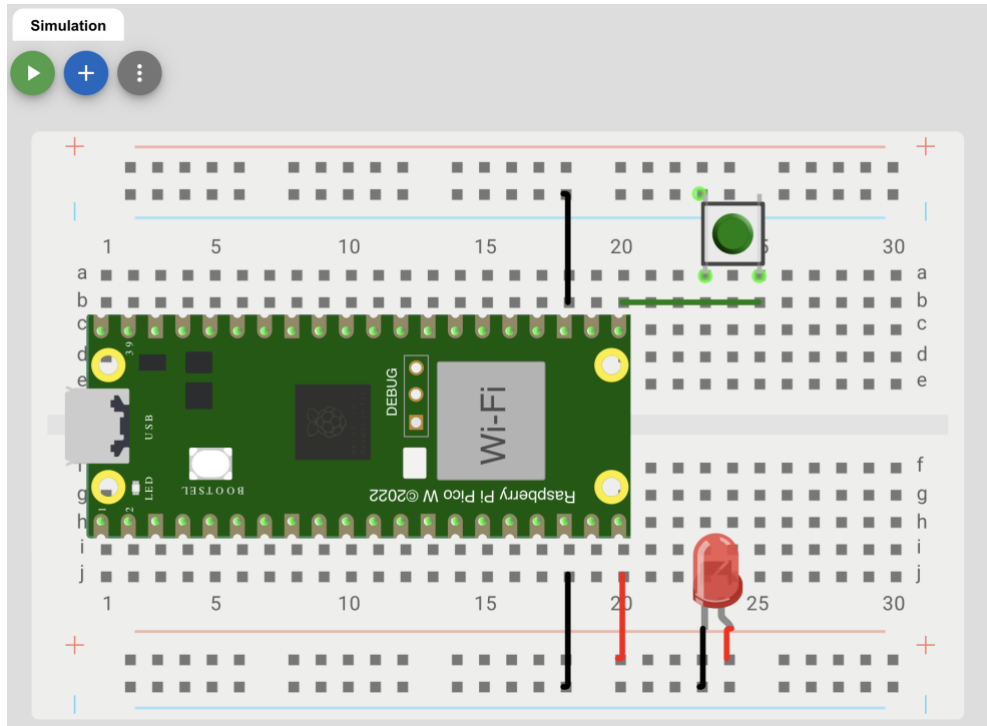
Materiales

1. Raspberry Pi Pico 2 W
2. Protoboard
3. LED rojo
4. Push button
5. Jumper

Software: Visual Studio Code

FSM — Debounce de botón y LED





Desarrollo

Parte 1: Antirrebote por FSM de botón/LED usando SDK

i. Código escrito

```

1  #include <stdio.h>
2  #include "pico/stdlib.h"
3
4  // Definición de pines y tiempo de antirrebote
5  #define BOTON 16
6  #define LED 15
7  #define VENTANA_MS 20
8
9  // Estados utilizados por la máquina de estados finitos
10 typedef enum {
11     ESTABLE_ALTO, // Botón liberado de forma estable
12     POSIBLE_BAJO, // Se detecta una posible pulsación
13     ESTABLE_BAJO, // Botón presionado de forma estable
14     POSIBLE_ALTO, // Se detecta una posible liberación
15 } estado_t;
16
17 int main(void)
18 {
19     // Configuración del botón como entrada con resistencia pull-up
20     gpio_init(BOTON);
21     gpio_set_dir(BOTON, false);
22     gpio_pull_up(BOTON);
23
24     // Configuración del LED como salida
25     gpio_init(LED);
26     gpio_set_dir(LED, true);
27
28     // Estado inicial: el botón se considera liberado
29     estado_t estado = ESTABLE_ALTO;
30
31     // Variable para almacenar el tiempo en que se detecta un cambio
32     absolute_time_t t_cambio;
33
34     while (true) {
35         // Lectura del nivel lógico del botón
36         // Alto = botón liberado, Bajo = botón presionado
37         bool nivel_alto = gpio_get(BOTON);
38
39         // Máquina de estados para realizar el antirrebote
40         switch (estado) {
41             case ESTABLE_ALTO:
42                 // Si el botón cambia a nivel bajo,
43                 // se inicia la validación de una posible pulsación
44                 if (!nivel_alto) {
45                     estado = POSIBLE_BAJO;
46                     t_cambio = get_absolute_time();
47                 }
48                 break;
49
50             case POSIBLE_BAJO:
51                 // Si vuelve a nivel alto antes de completar la ventana,
52                 // el cambio se considera un rebote
53                 if (nivel_alto) {
54                     estado = ESTABLE_ALTO;
55                 }
56                 // Si permanece en bajo por más de 20 ms,
57                 // se confirma la pulsación
58                 else if (absolute_time_diff_us(
59                     t_cambio,
60                     get_absolute_time()) > VENTANA_MS * 1000) {
61                     estado = ESTABLE_BAJO;
62                 }
63                 // Invierte el estado actual del LED
64                 gpio_xor_mask(1u << LED);
65                 break;
66
67             case ESTABLE_BAJO:
68                 // Si el botón cambia a nivel alto,
69                 // se inicia la validación de su liberación
70                 if (nivel_alto) {
71                     estado = POSIBLE_ALTO;
72                     t_cambio = get_absolute_time();
73                 }
74                 break;
75
76             case POSIBLE_ALTO:
77                 // Si vuelve a nivel bajo antes de completar la ventana,
78                 // se considera un rebote
79                 if (!nivel_alto) {
80                     estado = ESTABLE_BAJO;
81                 }
82                 // Si permanece en alto por más de 20 ms,
83                 // se confirma la liberación del botón
84                 else if (absolute_time_diff_us(
85                     t_cambio,
86                     get_absolute_time()) > VENTANA_MS * 1000) {
87                     estado = ESTABLE_ALTO;
88                 }
89                 // Invierte nuevamente el estado del LED
90                 gpio_xor_mask(1u << LED);
91                 break;
92         }
93     }
94 }

```

Parte 2: Antirrebote por FSM de botón/LED usando SIO

i. Código escrito

```

1 #include <stdio.h>
2 #include "pico/stlib.h"
3
4 // Definición de pines utilizados
5 #define BOTON 16
6 #define LED 15
7
8 // Tiempo que debe mantenerse estable una señal
9 // para considerarla válida y eliminar el rebote
10 #define VENTANA_MS 20
11
12 // Estados de la máquina de estados finitos
13 typedef enum {
14     ESTABLE_ALTO, // Botón liberado de forma estable
15     POSIBLE_BAJO, // Posible pulsación del botón
16     ESTABLE_BAJO, // Botón presionado de forma estable
17     POSIBLE_ALTO, // Posible liberación del botón
18 } estado_t;
19
20 int main(void)
21 {
22     // Inicializa el GPIO correspondiente al botón
23     gpio_init(BOTON);
24
25     // Limpia el bit correspondiente en GPIO_OE,
26     // configurando el GPIO16 como entrada
27     sio_hw->gpio_oe_clr = 1u << BOTON;
28
29     // Activa la resistencia pull-up interna.
30     // Por lo tanto:
31     // nivel alto = botón liberado
32     // nivel bajo = botón presionado
33     gpio_pull_up(BOTON);
34
35     // Inicializa el GPIO correspondiente al LED
36     gpio_init(LED);
37
38     // Coloca en 1 el bit correspondiente en GPIO_OE,
39     // configurando el GPIO15 como salida
40     sio_hw->gpio_oe_set = 1u << LED;
41
42     // La FSM inicia suponiendo que el botón está liberado
43     estado_t estado = ESTABLE_ALTO;
44
45     // Variable utilizada para guardar el instante
46     // en que se detecta un posible cambio
47     absolute_time_t t_cambio;
48
49     // Ciclo principal del programa
50     while (true) {
51         // Lee directamente el registro GPIO_IN,
52         // La máscara selecciona únicamente el bit correspondiente al botón.
53         bool nivel_alto = (sio_hw->gpio_in & (1u << BOTON));
54
55         // Máquina de estados para implementar
56         // el antirrebote del botón
57         switch (estado) {
58             case ESTABLE_ALTO:
59                 // Si se detecta un nivel bajo,
60                 // puede tratarse de una pulsación
61                 if (!nivel_alto) {
62                     // Se comienza a validar el cambio
63                     estado = POSIBLE_BAJO;
64                     // Guarda el tiempo en que inició el cambio
65                     t_cambio = get_absolute_time();
66                     break;
67                 }
68             case POSIBLE_BAJO:
69                 // Si vuelve a nivel alto antes de completar
70                 // los 20 ms, se considera un rebote
71                 if (nivel_alto) {
72                     estado = ESTABLE_ALTO;
73                     // Si permanece en nivel bajo durante más
74                     // de 20 ms, se confirma la pulsación
75                     } else if {
76                         absolute_time_diff_usf
77                         t_cambio,
78                         get_absolute_time()
79                     } > VENTANA_MS * 1000) {
80                         estado = ESTABLE_BAJO;
81                         // Coloca en alto el GPIO15 y enciende el LED
82                         sio_hw->gpio_set = 1u << LED;
83                         break;
84                     }
85             case ESTABLE_BAJO:
86                 // Si aparece un nivel alto,
87                 // puede tratarse de la liberación del botón
88                 if (nivel_alto) {
89                     // Se comienza a validar la liberación
90                     estado = POSIBLE_ALTO;
91                     // Guarda el instante en que inició el cambio
92                     t_cambio = get_absolute_time();
93                     break;
94                 }
95             case POSIBLE_ALTO:
96                 // Si vuelve a nivel bajo antes de completar
97                 // los 20 ms, se considera un rebote
98                 if (!nivel_alto) {
99                     estado = ESTABLE_BAJO;
100                     // Si permanece en nivel alto durante más
101                     // de 20 ms, se confirma que el botón fue liberado
102                     } else if {
103                         absolute_time_diff_usf
104                         t_cambio,
105                         get_absolute_time()
106                     } > VENTANA_MS * 1000) {
107                         estado = ESTABLE_ALTO;
108                         // Coloca en bajo el GPIO15 y apaga el LED
109                         sio_hw->gpio_clr = 1u << LED;
110                         break;
111                     }
112         }
113     }
114 }

```

Resultados

Se obtuvo un circuito funcional integrado por la Raspberry Pi Pico 2 W, una protoboard, un push button, un LED rojo y 4 jumpers.

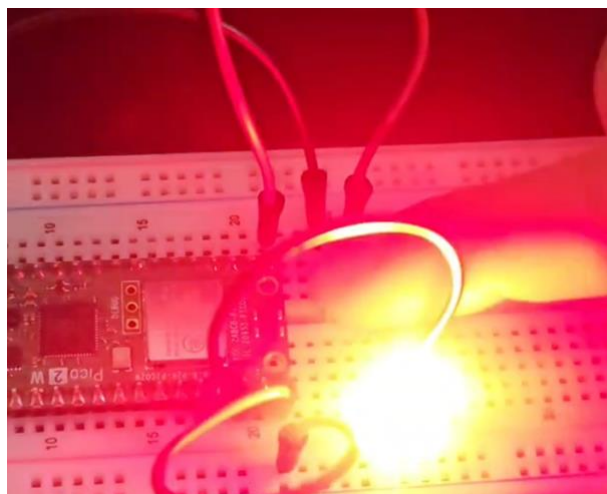
Evidencias

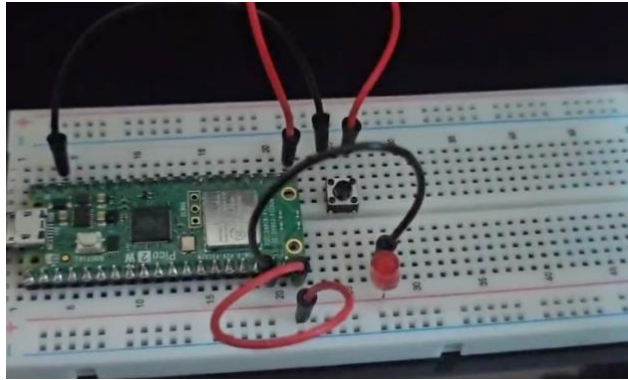
Simulación Wokwi SDK: <https://wokwi.com/projects/473730345739378689>

Simulación Wokwi SIO: <https://wokwi.com/projects/473731733652517889>

El siguiente enlace tiene una vigencia al 28 de Septiembre del 2026. Sin embargo, el archivo del video se puede encontrar dentro del zip adjuntado a la entrega de esta práctica.

https://iberopuebla-my.sharepoint.com/:v:/g/personal/201769_iberopuebla_mx/IQD6qwIQtxsxRLP00v6TUm9gAY6bgUB26Om8jVXYG4AYH-A?nav=eyJyZWZlcnJhbEluZm8iOnsicmVmZXJyYWxBcHAI0iJPbmVEcmI2ZUZvckJ1c2luZXNzIiwicmVmZXJyYWxBcHBQbGF0Zm9ybSI6IldlYiIsInJlZmVycmFsTW9kZSI6InZpZXciLCJyZWZlcnJhbFZpZXciOiJNeUZpbGVzTGlua0NvcHkifX0&e=D2CL8S





Conclusiones

Los resultados obtenidos demuestran el correcto funcionamiento de ambos códigos para la filtración de un rebote mecánico en el push button. A partir de la teoría se estableció que cualquier pulsación de un botón mecánico genera micro pulsaciones en un intervalo de 1 a 20 ms. Al ser mucho más rápida la velocidad del procesador para muestrear las entradas, estas micro pulsaciones pueden generar lecturas erróneas que derivarán en cambios de estados inexistentes provenientes de una sola pulsación.

De acuerdo con este planteamiento, los códigos utilizan la herramienta metodológica que máquina de estados finitos como solución antirrebote. Se establecieron cuatro estados (`ESTABLE_ALTO`, `POSIBLE_BAJO`, `ESTABLE_BAJO`, `POSIBLE_ALTO`) que determinan si las lecturas de entrada pertenecen a pulsaciones o son rebotes de esta. Debido a la configuración del botón, como pull up, si se presiona el boto la lógica será un 0 digital y al no presionarlo será un 1 digital. A partir de ello se determinó una validación de tiempo de 20ms para cada entre estado.

La evidencias adjuntas muestran que cada parpadeo del LED corresponde a la cantidad de pulsaciones realizadas, independientemente de la cantidad de tiempo que mantengamos presionado el botón. Sin embargo, la dificultad presentada durante dicha práctica recayó en la compatibilidad del LED. Debido a que se utilizó una Raspberry Pi Pico 2 W, su LED integrado no está conectado directamente a los periféricos especiales GPIO. Por ende, dicho microcontrolador sí puede utilizar máscaras, pero el LED integrado pertenece a un GPIO del chip CYW43 que necesita una forma diferente de acceso. Por lo cuál se prefirió utilizar el periférico GPIO para la implementación de un LED externo.

Declaración de IA

1. Claude
 - a. Búsqueda de fuentes para la realización del marco teórico
2. Lucid AI
 - a. Realización del esquemático FSM del código completo
3. ChatGPT
 - a. Comentado de códigos

Referencias

Raspberry Pi Ltd. (2025). RP2040 datasheet: A microcontroller by Raspberry Pi.
<https://datasheets.raspberrypi.com/rp2040/rp2040-datasheet.pdf>

Raspberry Pi Ltd. (2026). Raspberry Pi Pico-series C/C++ SDK: Libraries and tools for C/C++ development on Raspberry Pi microcontrollers.
<https://datasheets.raspberrypi.com/pico/raspberry-pi-pico-c-sdk.pdf>

Raspberry Pi Ltd. (2024). Getting started with Raspberry Pi Pico-series: C/C++ development with RP2040 and RP2350-based devices.
<https://datasheets.raspberrypi.com/pico/getting-started-with-pico.pdf>